

# Computation & Complexity Theory

Written by HADIOUCHE Azouaou.

## Disclaimer

This course has contents from multiple books that are stated in the reference, and not necessarily follow the school courses.

To separate the contents of the course to actual additions or out of context information, a black band will be added by its side like the one on this comment.

## Contents

|                                                                     |   |
|---------------------------------------------------------------------|---|
| <b>CHAPTER 1: INTRODUCTION</b>                                      | 2 |
| 1.1. Some History                                                   | 2 |
| 1.2. Goals Of The Course                                            | 2 |
| 1.3. Comments & Feedback                                            | 2 |
| <b>CHAPTER 2: COMPUTATION &amp; MACHINES</b>                        | 3 |
| 2.1. Machines & Finite Automata                                     | 3 |
| 2.1.1. Deterministic Automaton                                      | 3 |
| 2.1.2. Non-Deterministic Finite Automaton                           | 5 |
| 2.2. Regular Languages & Regular Expressions                        | 7 |
| 2.3. Efficiency Of Description                                      | 8 |
| <b>CHAPTER 3: TURING MACHINES &amp; UNIVERSALITY OF COMPUTATION</b> | 9 |
| The Birth Of Turing Machines                                        | 9 |
| Turing Machines                                                     | 9 |

|                                               |    |
|-----------------------------------------------|----|
| Two's Complement Inverter                     | 11 |
| Different Definitions Of Turing Machines      | 12 |
| Undecidability                                | 12 |
| Uncomputable Functions                        | 14 |
| Church-Turing Thesis                          | 14 |
| Universal Turing Machines                     | 14 |
| Modern Programming Languages                  | 14 |
| Turing Completeness                           | 16 |
| Universal Machines & Their Existence          | 16 |
| Decidability & Computability                  | 17 |
| Types Of Turing Machines                      | 17 |
| Deterministic Turing Machine                  | 17 |
| Non-Deterministic Turing Machine              | 17 |
| Complexity & Runtimes                         | 19 |
| Classes Of Complexity                         | 19 |
| Deterministic Polynomial Class <b>P</b>       | 19 |
| Non-Deterministic Polynomial Class <b>NP</b>  | 20 |
| NP-Complete Class <b>NP</b>                   | 21 |
| Conclusion                                    | 22 |
| <b>CHAPTER 4: COMPLEXITY &amp; ALGORITHMS</b> | 23 |
| <b>REFERENCES</b>                             | 24 |

# Chapter 1

## Introduction

Computation & complexity theory are two theories that focus on the possibility and the optimality of computation. They both study the aspect of computing objects and algorithms in two different ways: computation theory answers the questions that are related to the existence of algorithms and procedures, while complexity theory answers the questions that are related to the resources used by algorithms and theoretical limits for such usages.

### 1.1. Some History

The foundations of computation theory started around the 20<sup>th</sup> century, as a mathematical question to understand the limits of mathematical reasoning. David Hilbert at the time had questions about general mathematical truths about the used logic fundamentals: are mathematics complete, consistent and decidable.

In 1936, multiple models for computation have been born, Alonzo Church's  $\lambda$ -calculus, Alan Turing's Turing machines and Emil Post's Post machine. Those machines have really different descriptions, while are proven to be equivalent, that is, any computation doable in the one, it can be done in the others. Such proof lead to the Church-Turing thesis, which states in principle any computation can be done by a Turing machine.

Turing proved the existence of problems that cannot be solved by algorithms. One of such problems is the Halting Problem showing a clear limit for computation and giving rise to the theory of computability.

As the technology kept improving and computers became more practical in usual tasks around the 1950s, a new question became the focus of computer scientists, how efficient an algorithm for a task can be. Multiple algorithms can solve the same problem, but their efficiency is usually different, so, what is the efficiency limit for how can these be computed.

The focus of such discipline turned after Stephen Cook introduced the concept of NP-completeness in 1971, and proving that the SAT problem is NP-complete. Since then, the  $P \stackrel{?}{=} NP$  question became a central open question in theoretical computer science which is unsolved until now (2026).

### 1.2. Goals Of The Course

In this course, our goal is to introduce as much concepts that were talked about here as possible, giving the mathematical foundation for computer science and goes through most questions that are related to these two beautiful theories.

Also, there will be multiple demonstrations and examples that will be available in my GitHub page, have fun, try discovering things. There will be also exercises and problem sheets taken from books and sources that would give a broader view on the domain, it would be my pleasure to check and help as much as possible.

### 1.3. Comments & Feedback

Please contact me in case you have any comments, questions, or need help in any part of the course. There will also be a page for displaying feedback and comments on the courses, which will be updated from every now and then.

## Chapter 2

# Computation & Machines

As discussed previously, we start by trying to define what is computation, to do this, we will go through some models of computation and see their power and limits until we arrive to the most powerful machines we know.

**Notation:** Let  $\Sigma$  be a set of alphabets. We define the following:

- $\mathbb{N} = \{0, 1, 2, \dots\}$  and for  $n < m$ ,  $\llbracket n, m \rrbracket = \{n, n+1, \dots, m-1, m\}$ .
- $\Sigma^n = \Sigma \times \dots \times \Sigma$  the set of all words written with  $n$  alphabets of  $\Sigma$ , for  $n = 0$ , we get  $\Sigma^0 = \{\varepsilon\}$  where  $\varepsilon$  is the empty word.
- If  $\Sigma = \{a\}$  then  $\Sigma^n = \{a\}^n$  and we denote it  $a^n$ .
- $\Sigma^* = \bigcup_{n \in \mathbb{N}} \Sigma^n$  the set of all possible words written with alphabets of  $\Sigma$ .
- Instead of using the notation  $x = (x_1, x_2, \dots, x_n)$  we will use the word notation  $x = x_1x_2\dots x_n$ .
- If  $x = x_1x_2\dots x_n \in \Sigma^n$ ,  $y = y_1y_2\dots y_m \in \Sigma^m$ , we define the concatenation operation  $xy$  such that  $xy = x_1x_2\dots x_ny_1y_2\dots y_m \in \Sigma^{n+m}$  and for  $S_1, S_2$  two sets of words, we define  $S_1S_2 = \{xy \mid x \in S_1, y \in S_2\}$ .
- Define the length  $|\cdot| : \Sigma^* \rightarrow \mathbb{N}$ ,  $S \mapsto |S| = \min\{n \in \mathbb{N} \mid S \in \Sigma^n\}$ .

## 2.1. Machines & Finite Automata

We start by defining the most basic computation machines that have almost no memory to have an idea of what are computation models.

### 2.1.1. Deterministic Automaton

**Definition 1 (DFA):** Let  $M = (\Sigma, Q, \delta, q_0, F)$  a deterministic finite automaton (DFA), where

- $\Sigma$  an alphabet.
- $Q$  a finite set of states.

- $\delta : Q \times \Sigma \rightarrow Q$  a transition function.
- $q_0 \in Q$  a starting state.
- $F \subseteq Q$  a set of accepted/final states.

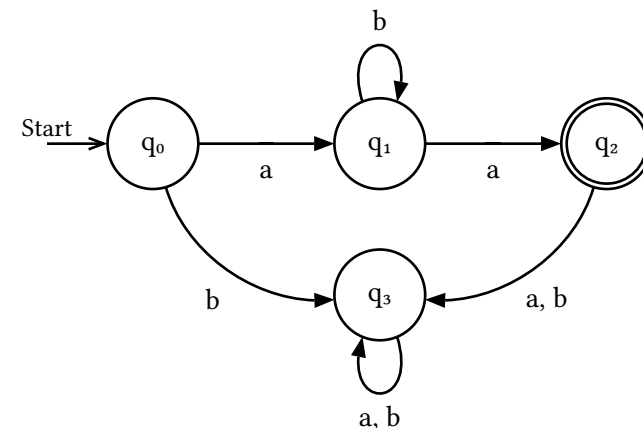
A computation in  $M$  is done as follows: let  $S = s_1s_2\dots s_m \in \Sigma^*$ , define the sequence  $\{D_i\}_{i \in \llbracket 1, m \rrbracket} \subseteq Q$  that satisfies the recursion

$$\begin{cases} D_0 = q_0 \\ D_i = \delta(D_{i-1}, s_i) \text{ with } i \in \llbracket 1, m \rrbracket \end{cases}$$

$M$  is said to accept  $S$  if and only if  $D_m \in F$ , it is said to be rejected otherwise. We define the language of  $M$  denoted  $\mathcal{L}(M)$  as the set of all strings in  $\Sigma^*$  that the machine  $M$  accepts. And denote the set of all languages that are recognizable by a DFA as  $\mathcal{L}_{\text{DFA}}$ .

Note that the finite in DFA comes from the fact that the set of states is finite. We have a natural way to represent DFAs as a graph which is similar to discrete Markov chains. We take the following example: Let  $M = (\Sigma, Q, \delta, q_0, F)$  with  $\Sigma = \{a, b\}$ ,  $Q = \{q_0, q_1, q_2, q_3\}$  and  $F = \{q_3\}$  and the transition function is the table on the right. It is represented as follows.

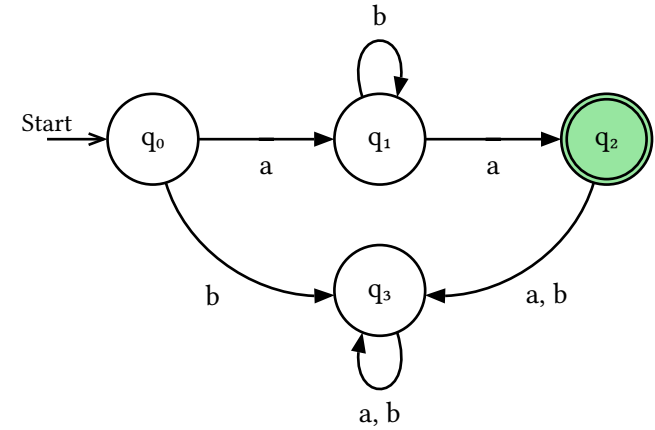
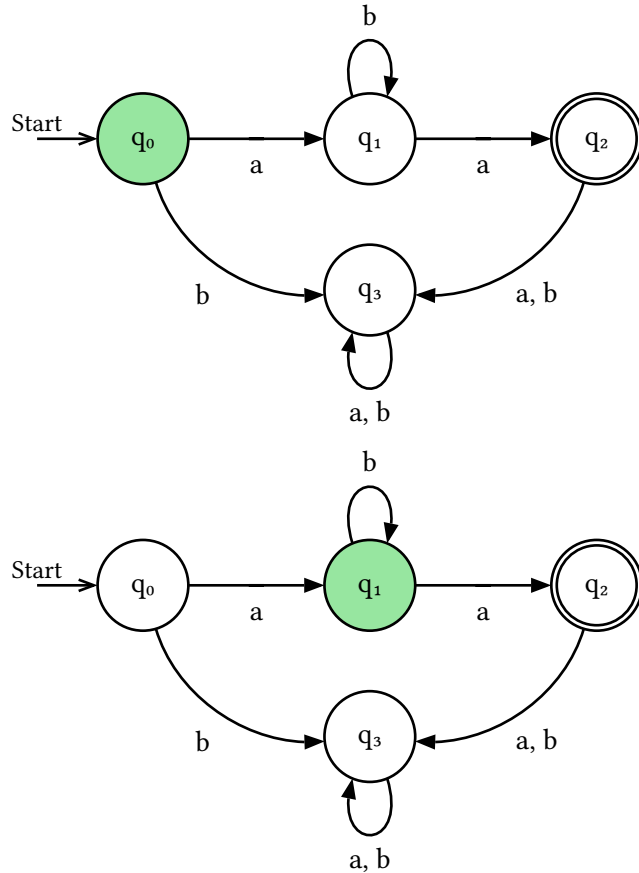
| $q \in Q$ | $c \in \Sigma$ | $\delta(q, c)$ |
|-----------|----------------|----------------|
| $q_0$     | $a$            | $q_1$          |
| $q_0$     | $b$            | $q_2$          |
| $q_1$     | $a$            | $q_2$          |
| $q_1$     | $b$            | $q_1$          |
| $q_2$     | $a, b$         | $q_3$          |
| $q_3$     | $a, b$         | $q_2$          |



We will run it on the following string  $S = abba$

- $D_0 = q_0, S \leftarrow abba$
- $D_1 = \delta(D_0, s_1) = \delta(q_0, a) = q_1, S \leftarrow bba$
- $D_2 = \delta(D_1, s_2) = \delta(q_1, b) = q_1, S \leftarrow ba$
- $D_3 = \delta(D_2, s_3) = \delta(q_1, b) = q_1, S \leftarrow a$
- $D_4 = \delta(D_3, s_4) = \delta(q_1, a) = q_2, S \leftarrow \varepsilon$

Notice that  $D_4 = q_2 \in F$  thus  $S \in \mathcal{L}(M)$ , we will represent these transitions using the machine and we will get the same result.



It is easy to notice that  $\mathcal{L}(M) = \{ab^k a \mid k \in \mathbb{N}\}$ , we will prove it

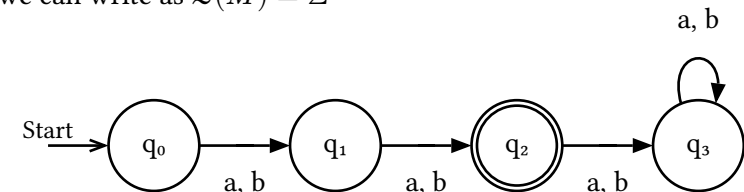
**Proposition 2:** Let  $M$  the defined automaton in the previous example, then  $\mathcal{L}(M) = \{ab^k a \mid k \in \mathbb{N}\}$ .

*Proof.* Let  $S = s_1 s_2 \dots s_n \in \mathcal{L}(M)$  and  $D_i$  the associated sequence of steps.

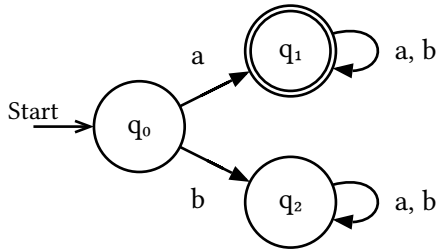
- $s_1 = a$ : suppose by contradiction that  $s_1 = b$ , then  $D_1 = \delta(D_0, b) = q_3$  thus we get  $\forall i \in \llbracket 1, n \rrbracket, D_i = q_3$  but  $q_3 \notin F$ , so  $s_1 = a$ .
- $\forall i \in \llbracket 2, n-1 \rrbracket, s_i = b$ : since  $s_1 = a$  then  $D_2 = q_1$ , if for some  $i \in \llbracket 2, n-1 \rrbracket, s_i = a$  then  $D_i = q_2$  thus  $D_{i+1} = q_3$  and using the same argument as before we get that  $q_3 \notin F$  which is a contradiction, thus  $\forall i \in \llbracket 2, n-1 \rrbracket, s_i = b$ .
- $s_n = a$ : notice that  $D_{n-1} = q_1$  thus if  $s_n = b$ ,  $D_n = q_1 \notin F$  hence  $s_n = a$ . ■

**Example:** We will go through a series of examples of automata and their languages. We consider  $\Sigma = \{a, b\}$ .

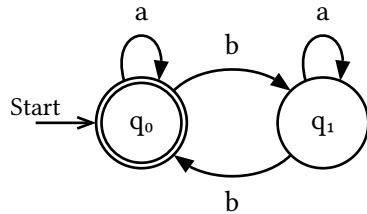
1. The language of the automaton below is the set of words with length exactly 2 which we can write as  $\mathcal{L}(M) = \Sigma^2$



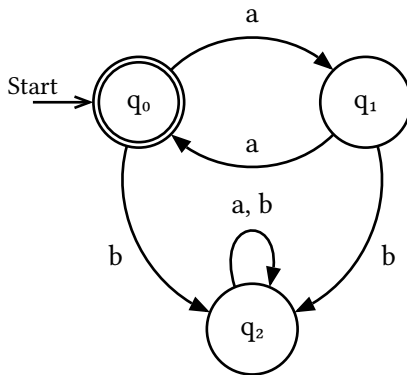
2. The language of the automaton below is the set of words starting with  $a$ , written as  $\mathcal{L}(M) = a\Sigma^* = \{aw \mid w \in \Sigma^*\}$ .



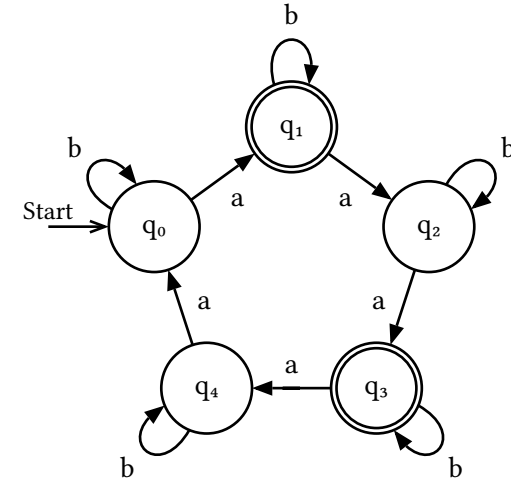
3. The language of the automaton below is the set of words that have an even number of  $b$ s,  $\mathcal{L}(M) = \{S \in \Sigma^* \mid \#b(S) \text{ is even}\}$ , notice in this case the  $\varepsilon \in \mathcal{L}(M)$  since  $\#b(\varepsilon) = 0$  is even too.



4. The language of the automaton below is the set of words that are of the form  $aa, aaaa, aaaaaa, \dots, a^{2n}$ ,  $\mathcal{L}(M) = \{a^{2n} \mid n \in \mathbb{N}\}$ .



5. The language of the automaton below is the set of words that have either 1 or  $3 \bmod 5$   $a$ s,  $\mathcal{L}(M) = \{S \in \Sigma^* \mid (\#a(S) \bmod 5) \in \{1, 3\}\}$ .



The expression of such a machine is really limited, that is, if the set of languages  $\mathcal{L} \subseteq \Sigma^*$  that deterministic finite automata can determine is really limited, the set of languages that the deterministic finite automata can detect is called regular languages. An example of a language that is not regular is  $\mathcal{L} = \{a^n b^n \mid n \in \mathbb{N}\}$  where  $\Sigma = \{a, b\}$ .

### 2.1.2. Non-Deterministic Finite Automaton

A non-deterministic automaton is a generalization of the automaton, where we give it more abilities to be stronger than the deterministic finite automata.

**Definition 3 (NFA):** Let  $M = (\Sigma, Q, \delta, q_0, F)$  a non-deterministic finite automaton (NFA or NFA), it satisfies:

- $\Sigma$  an alphabet.
- $Q$  a finite set of states.
- $\delta : Q \times (\Sigma \cup \{\varepsilon\}) \rightarrow \mathcal{P}(Q)$ .
- $q_0$  a starting state.
- $F \subseteq Q$  a set of accepted/final states.

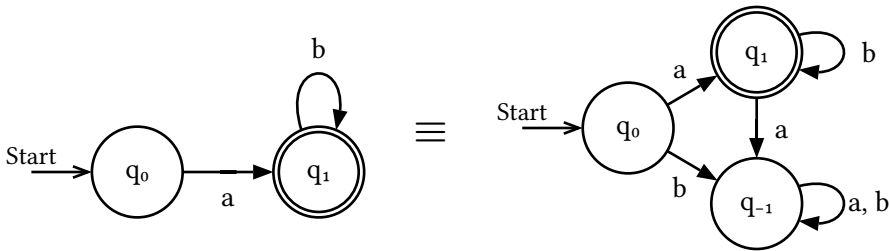
A computation in  $M$  is done as follows: let  $S = s_1 s_2 \dots s_m \in \Sigma^*$ ,  $S$  is said to be accepted by  $M$  if  $\delta^*(q_0, S) \cap F \neq \emptyset$ , where  $\delta^* : Q \times \Sigma^* \rightarrow \mathcal{P}(Q)$  is recursively defined as

$$\begin{cases} \forall q \in Q, \delta^*(q, \varepsilon) = E(\{q\}) \\ \forall q \in Q, \forall s \in \Sigma, \forall S \in \Sigma^*, \delta^*(q, Sx) = E\left(\bigcup_{q' \in \delta^*(q, S)} \delta(q', x)\right) \end{cases}$$

and  $E(R)$ , is defined as the set of all states that  $R$  can reach with using only  $\varepsilon$ -transitions, . We define the language of  $M$  denoted  $\mathcal{L}(M)$  as the set of all strings in  $\Sigma^*$  that the machine  $M$  accepts. And denote the set of all languages that are recognizable by an NFA as  $\mathcal{L}_{\text{NFA}}$ .

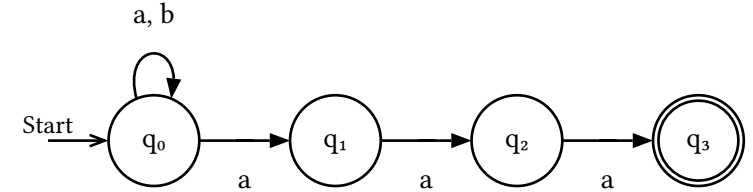
In this new model, we have 3 additional features:

1. The ability to jump without consuming any character from the string with the  $\varepsilon$ -transitions, that is, the transitions that have  $\varepsilon$  as the character.
2. The ability to jump to multiple states using just one letter of the string, this comes from the fact that  $\delta(q, c)$  is a set of possible paths to take.
3. The implicit transitions to trap states, that is, if  $\delta(q, c)$  is not defined, then we assume implicitly that it jumps to a state  $q_0$  and does not transition to anything else. To do this, we just create a new state  $q_{-1}$ , and whenever there is no transition from a state  $q_i$  with letter  $c$  then we define  $\delta(q_i, c) = \{q_{-1}\}$  and we define  $\forall c \in \Sigma \cup \{\varepsilon\}, \delta(q_{-1}, c) = \{q_{-1}\}$ .

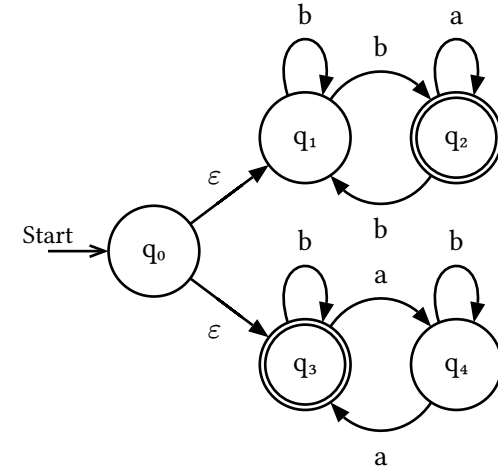


The non-determinism comes from the fact that we do not have a deterministic path we walk along, but each step may lead to many paths. We will go through a series of examples of NFAs to see how those features add expression.

1. The language of the automaton below is the set of words that end with 3 as, which we can write as  $\mathcal{L}(M) = \Sigma^*aaa$ . Notice that there are two transitions with  $a$  that go from  $q_0$ .



2. The language of the automaton below is the set of words that either have an odd number of bs or have an even number of as, which can be written as follows  $\mathcal{L}(M) = \{S \in \Sigma \mid \#a(S) \text{ is even} \vee \#b(S) \text{ is odd}\}$ .



From the previous examples, we can assume that an NFA will be stronger than an NFA, which is wrong. The set of languages that can be determined with an NFA is exactly the one that can be determined by a DFA. It is clear that any DFA is an NFA. Now we will prove that the languages from NFA can be determined using DFA too.

**Lemma 4:**  $E$  is idempotent,  $E \circ E = E$ .

*Proof.* Let  $q \in Q$  such that  $q \in E \circ E(R)$  and  $q \notin E(R)$ , since  $q \in E(E(R))$  then there exists  $q' \in E(R)$  such that  $q'$  and  $q$  have an  $\varepsilon$ -transition, but since  $q' \in E(R)$  then there exists an  $\varepsilon$ -transition from  $q'' \in R$  to  $q'$ . We have that  $q'' \xrightarrow{\varepsilon} q'$  and  $q' \xrightarrow{\varepsilon} q$  hence  $q'' \xrightarrow{\varepsilon} q$  hence  $q \in E(R)$ . ■

**Theorem 5:**  $\mathcal{L}_{\text{DFA}} = \mathcal{L}_{\text{NFA}}$ , that is, if  $M$  is an NFA, then there exists a DFA  $M'$  such that  $\mathcal{L}(M) = \mathcal{L}(M')$ .

*Proof.* Let  $M = (\Sigma, Q, \delta, q_0, F)$  be an NFA and  $E(R)$  the set of states accessible from  $R$  using only  $\varepsilon$ -transitions. We define the DFA  $M' = (\Sigma, Q', \delta', q'_0, F')$  with the parameters  $Q' = \mathcal{P}(Q)$ ,  $q'_0 = E(\{q_0\})$ ,  $F' = \{R \subseteq Q \mid R \cap F \neq \emptyset\}$ . And we define the transition function as follows

$$\forall R \in Q', \delta'(R, c) = E\left(\bigcup_{q \in R} \delta(q, c)\right)$$

Notice that in this case we have no  $\varepsilon$ -transitions, and the transition goes from one state to another only, and hence,  $M'$  is indeed just a deterministic finite automaton, now we prove the following claim:  $\forall S \in \Sigma^*, \delta'^*(q'_0, S) = \delta^*(q_0, S)$ . We prove it by induction from  $\delta^*$  recursive definition.

- $\delta'^*(q'_0, \varepsilon) = E(\{q'_0\}) = E(E(q_0)) = E(\{q_0\}) = \delta^*(q_0, \varepsilon)$
- Suppose by induction we have  $\delta'^*(q'_0, S) = \delta^*(q_0, S)$ , then we have

$$\begin{aligned} \delta'^*(q'_0, Sx) &= E\left(\bigcup_{q' \in \delta'^*(q'_0, S)} \delta'(q', x)\right) \\ &= E\left(\bigcup_{q' \in \delta^*(q'_0, S)} \delta(q', x)\right) \\ &= \delta^*(q_0, Sx) \end{aligned}$$

$$S \in \mathcal{L}(M) \Leftrightarrow \delta^*(q_0, S) \cap F \neq \emptyset \Leftrightarrow \delta'^*(q'_0, S) \cap F \neq \emptyset \Leftrightarrow S \in \mathcal{L}(M') \quad \blacksquare$$

## 2.2. Regular Languages & Regular Expressions

An interesting way to represent the set of languages  $\mathcal{L}_{\text{DFA}}$ , called the regular languages, are the regular expressions, which are useful in general in “search” aspects as it allows a good syntax for expressing general patterns.

**Definition 6 (Regular Languages):** A language  $\mathcal{L} \subseteq \Sigma^*$  is said to be regular if  $\mathcal{L} \in \mathcal{L}_{\text{DFA}}$  (or  $\mathcal{L}_{\text{NFA}}$  since they are the same).

**Definition 7 (Regular Expression):** Let  $\Sigma$  be a set of alphabet, a regular expression is defined recursively as follows:

- $\varepsilon$  is a regular expression.
- for every symbol  $c \in \Sigma$ ,  $c$  is a regular expression.
- for every two regular expressions  $e, e'$  then
  - $ee'$  is a regular expression.
  - $e|e'$  is a regular expression.
  - $e^n$  is a regular expression.
  - $e^*$  is a regular expression.

The language of a regular expression  $e$ , denoted  $\mathcal{L}(e)$  is a subset of  $\Sigma^*$  defined recursively as follows

- $\mathcal{L}(\varepsilon) = \{\varepsilon\}$ .
- for any symbol  $c \in \Sigma$ ,  $\mathcal{L}(c) = \{c\}$ .
- for any expressions  $e, e'$ ,  $\mathcal{L}(ee') = \mathcal{L}(e)\mathcal{L}(e')$ .
- for any expressions  $e, e'$ ,  $\mathcal{L}(e|e') = \mathcal{L}(e) \cup \mathcal{L}(e')$ .
- for any expression  $e$ ,  $\mathcal{L}(e^n) = \mathcal{L}(e)^n$ .
- for any expression  $e$ ,  $\mathcal{L}(e^*) = \mathcal{L}(e)^*$ .

We consider some basic examples of regular expressions.

- $e = 0^*1^*$ , we obtain the following language  $\mathcal{L}(e) = \mathcal{L}(0^*1^*) = \mathcal{L}(0^*)\mathcal{L}(1^*) = 0^*1^*$ , notice that sometimes the expression and its language are just the same.
- $e = (01)^*$ , we have that the language of this expression is as follows

$$\mathcal{L}(e) = \mathcal{L}((01)^*) = \mathcal{L}(01)^* = (\mathcal{L}(0)\mathcal{L}(1))^* = (01)^*.$$

and hence the language of  $e$  is  $\mathcal{L}(e) = \{\varepsilon, 01, 0101, 010101, \dots\}$ .

- $e = (\Sigma\Sigma)^*$ , we have that the language of this expression is clearly  $(\Sigma\Sigma)^*$ , the set of words of even length.
- suppose  $\Sigma = \mathcal{A} \cup \mathcal{a} \cup \mathcal{N} \cup \mathcal{P}$  where

$$\begin{aligned}\mathcal{A} &= \{\mathbf{A}, \dots, \mathbf{Z}\} & \mathcal{a} &= \{\mathbf{a}, \dots, \mathbf{z}\} \\ \mathcal{N} &= \{0, \dots, 9\} & \mathcal{P} &= \{., @\}\end{aligned}$$

we can select all texts that are email accounts using regular expressions, the expression would be as follows

$$e = (\mathcal{A}|\mathcal{a})(\mathcal{A}|\mathcal{a}|\mathcal{N})^* @ (\mathcal{a}^*.)^* \mathcal{a}$$

the expression here expresses the following:

- the email address should start with a letter (uppercase or lowercase).
- the email address can contain as many letters or numbers after the first letter.
- the email will contain an @ to separate the name from the organization.
- the organization may have multiple suborganizations separated by points.

And hence the following strings are in the language of  $e$

$$\begin{aligned}\text{had.azouaou@gmail.com} &\in \mathcal{L}(e) \\ \text{azouaou.hadiouche@nhsm.edu.dz} &\in \mathcal{L}(e)\end{aligned}$$

Regular expressions, as their name suggests, have regular languages as a language. But also, any regular language has a regular expression representing it, the proof of such theorem is a bit long and technical, using generalized finite automata, hence we leave it for the student to explore and try to prove.

**Theorem 8:** A language  $\mathcal{L}$  is regular if and only if there exists a regular expression  $e$  such that  $\mathcal{L} = \mathcal{L}(e)$ .

## 2.3. Efficiency Of Description

In these very basic two models of computation, NFA and DFA, we now that the expression power is the same, yet, it is easy to see that an NFA needs less states than a DFA to achieve the same result.

Consider the following example of an NFA

## Chapter 3

# Turing Machines & Universality Of Computation

Since the beginning of history, computations were an important part of the human daily life. From the simple finance calculations for price predictions in the old times, to the huge large language models we have now. Throughout history, multiple models, and ways to describe computation machines came. But the one that is most used is the Turing machine.

In the 20<sup>th</sup> century, mathematics went through trouble of setting foundations, after some unintuitive results like the existence of multiple infinities by Cantor, and contradictory sets by Russel. By then, multiple mathematicians, mainly Hilbert, set out proposed to formalize all mathematics in a complete, decidable and consistent axiomatic system. The questions more specifically were:

1. Completeness: can every true statement be proven.
2. Consistency: are all statements free of contradiction.
3. Decidable: is there a procedure to determine truth.

Turing set out to find an answer for the decidability problem, and thus had to find a formal definition to what a “procedure” is, so he imagined an idealized “computer” that follows fixed rules, and reduced the computation to the following components: a tape, a pointer that reads and writes on the tape, and a finite set of instructions. Turing was able to prove that no axiomatic system would be decidable algorithmically. And it opened a gate to the study of computations also complexity after.

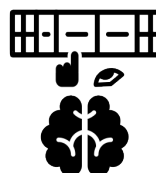
Beginning by talking about an intuitive way to create a machine that does exactly what a human does when stating or proving theorems, same as the situation

Turing thought of in the create of those machines. To think of a machine that can do this, it needs 3 components:

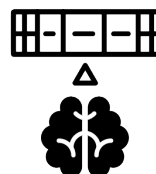
1. An ability to read the inputs.
2. An ability to write the outputs.
3. An ability to process the data.



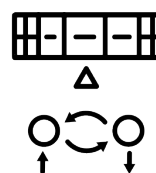
Consider an idealized human, or the parts that we need for describing a process. It needs a paper to be able to write, read and register the intermediate steps, as well as an eye and pen to be able to do these operations, and also a brain that does the processing.



Texts, books or any paper come in multiple different shapes, sizes, but the idea is the same, thus we abstract it into a tape, which may have any length possible and describes the data in symbols on the cells of the tape.



We can abstract the ability to read and write to a pointer at a position on the tape, which does actually act exactly like a hand and eye would do. And makes it less cumbersome to have just a single position to do the work at.



The human mind while in a process passes through multiple steps, each can be represented by a state-machine, that given inputs and intermediate steps in the process, may change to other states to continue the process.

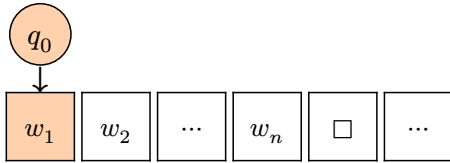
**Definition (Turing Machine):** A Turing machine  $M$  is a tuple  $(Q, \Sigma, \Gamma, \delta, q_0, q_a, q_r)$  where:

- $Q$ : finite set of states.
- $\Sigma$ : finite alphabet for transitions.
- $\Gamma$ : finite alphabets for tape with  $\square \in \Gamma \setminus \Sigma$ .
- $q_0, q_a, q_r \in Q$ : the start, accept, and reject states.
- $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$ : a transition function which does the “processing”.

Turing machines may be defined in many other ways, many books or lectures take the freedom of choosing the one that works most for them, for example here are some other different definitions:

- $M$  has multiple tapes for input, for work, for output.
  - $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, S, R\}$ , where  $S$  is a state jump without the head of the pointer moving.
  - $\Sigma = \{0, 1\}$  thus representing all the states we want using some binary encoding.
- After working with some Turing machines, the reader can easily prove that those are equivalent definitions. Now that we defined Turing machines, we will define what a computation is in said machine.

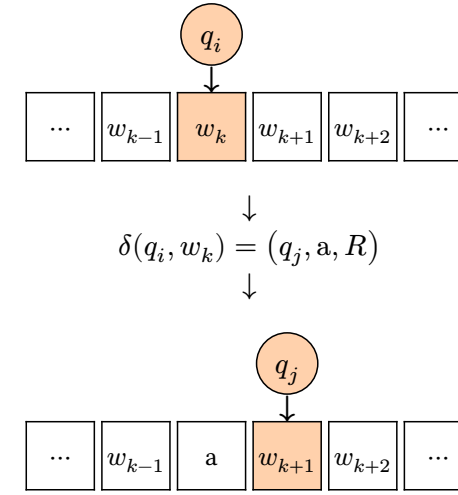
1. Start with the tape having  $w_1, \dots, w_n, \square, \square, \dots \rightarrow \infty$  and with the state machine being at  $q_0$ .



2. At each step, we are at a state  $q_i$ , and the pointer is on the  $p_i^{\text{th}}$  position, we evaluate the transition function to get the following

$$\delta(q_i, c) = (q_j, c', N)$$

with  $q_i, q_j \in Q, c, c' \in \Gamma, N \in \{L, R\}$  the instruction here is “if you are at state  $q_i$  and the pointer points at the symbol  $c$ , then change the state to  $q_j$ , change the symbol at the current position in the tape to  $c'$  and then move the pointer  $N$  (either right for  $R$  or left for  $L$ )”.



3. If at any step,  $q_i = q_a$  or  $q_i = q_r$ , the computation is finished, the result will be on the tape and the result is accepted if  $q_i = q_a$  and rejected if  $q_i = q_r$ .

Notice that nothing in this process forces the computation to terminate, and thus, there may exist infinite loops that never reject nor accept, we will talk about this more in the section of undecidability.

We define the language of a Turing machine  $M$  the set  $\mathcal{L}(M) = \{w \in \Sigma^* \mid M \text{ accepts } w\}$ . We start by creating some Turing machines for some known algorithms to see in practice how they work. Below are some useful notations that we may need later in the proofs.

#### Notation:

- $\Sigma^*$  is the set of all possible words written with  $\Sigma$ .
- $\triangleright$  A word  $w$  is any element of  $\Sigma^*$ .
  - $\triangleright w = w_1 \dots w_n$  represents  $w = (w_1, \dots, w_n)$ .
  - $\triangleright w = (w_i), v = (v_i) \Rightarrow wv = w_1 \dots w_n v_1 \dots v_m$ .
- $\triangleright$  A language  $\mathcal{L}$  is any subset of  $\Sigma^*$ .
  - $\triangleright \mathcal{L}_1, \mathcal{L}_2 \subseteq \Sigma^*, \mathcal{L}_1 \mathcal{L}_2 = \{vw \mid v \in \mathcal{L}_1, w \in \mathcal{L}_2\}$ .
- $\triangleright \langle x \rangle$  is a canonical representation of the mathematical object  $x$  in the language.
  - $\triangleright \langle (x_1, \dots, x_n) \rangle = \langle x_1, \dots, x_n \rangle$ .

▸  $|x|$  represents the length of the representation of the mathematical object  $x$  in the language.

Let  $K = k_n \dots k_0$  be the two's complement representation of a number  $x$ , to get the representation of  $-x$ , we have an easy algorithm:

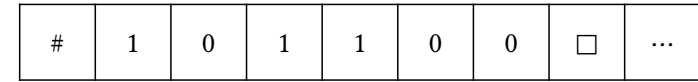
1. Start from right to left.
2. Move until first 1, denote it  $k_i$ .
3. Flip all bits after  $k_i$ .

To create the Turing machine, we specify all the parts given in the definition.

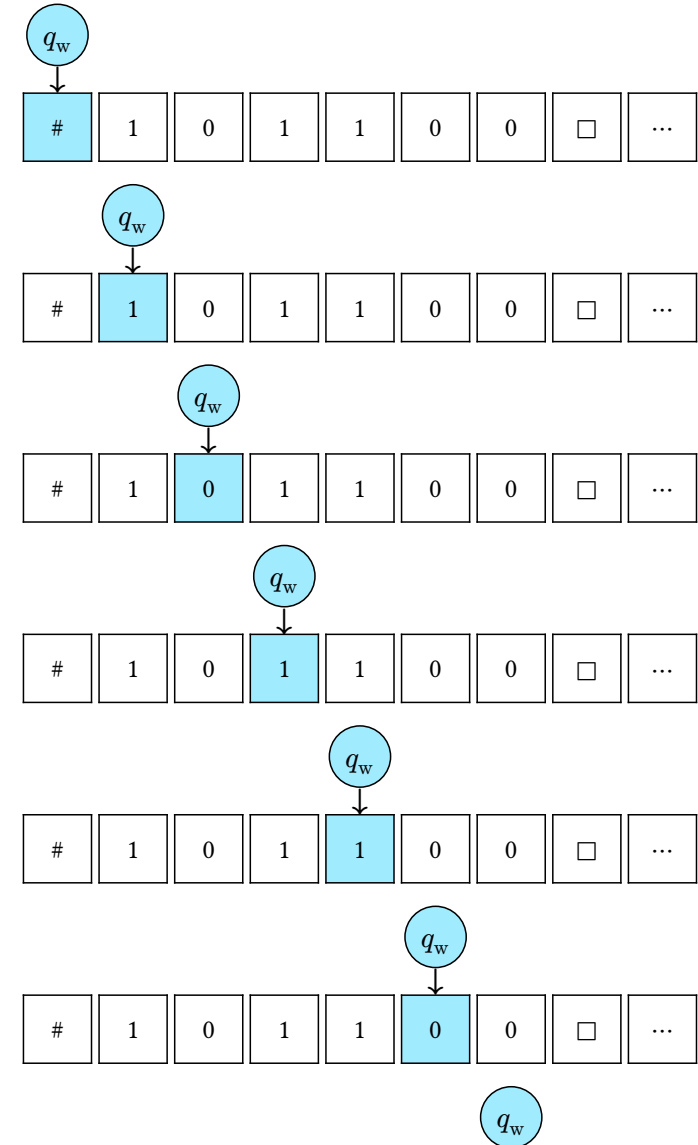
- $Q = \{q_w, q_s, q_f, q_d, q_r\}$ .
- $\Sigma = \{0, 1, \#\}$ .
- $\Gamma = \{0, 1, \#, \square\}$ .
- $(q_0, q_a, q_r) = (q_w, q_d, q_r)$ .
- $\delta$  is defined in the table below

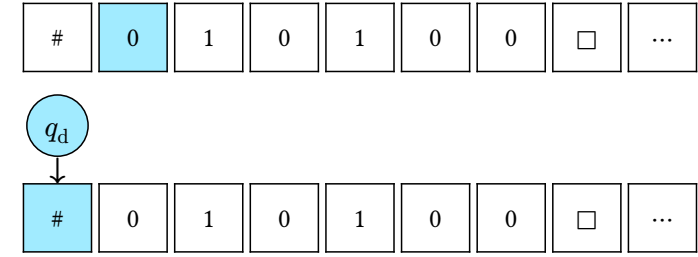
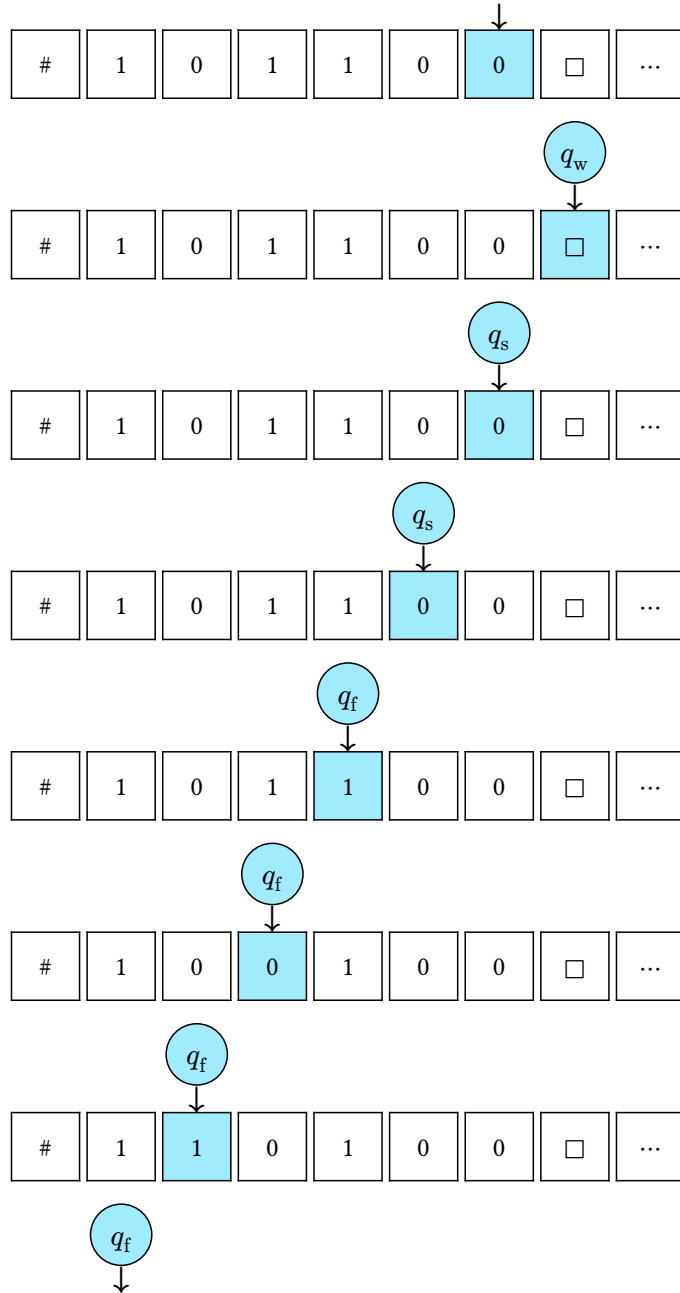
| $q \in Q$ | $c \in \Gamma$ | $\delta(q, c) \in Q \times \Gamma \times \{L, R\}$ |
|-----------|----------------|----------------------------------------------------|
| $q_w$     | 0, 1, #        | $(q_w, c, R)$                                      |
| $q_w$     | $\square$      | $(q_s, \square, L)$                                |
| $q_s$     | 0              | $(q_s, 0, L)$                                      |
| $q_s$     | 1              | $(q_f, 1, L)$                                      |
| $q_s$     | #              | $(q_d, \#, L)$                                     |
| $q_s$     | $\square$      | $(q_r, \square, L)$                                |
| $q_f$     | 0              | $(q_f, 1, L)$                                      |
| $q_f$     | 1              | $(q_f, 0, L)$                                      |
| $q_f$     | #              | $(q_d, \#, L)$                                     |
| $q_f$     | $\square$      | $(q_r, \square, L)$                                |

Here we just take the first element of the tape is # to specify that end of the binary representation string. We simulate it now, we start by preparing the tape and the pointer. Take the number  $K = 101100$  which represents  $-20$ . Here is how the tape would be shown



We start with the pointer at position 0, moves until it arrives to the space character, and then it starts “processing” the input on the tape.





Reading from the tape we get that the representation of the inverse is 010100 which is 18. It is easy to notice that the language of this machine is  $\mathcal{L}(M) = \# \{0, 1\}^* \square^{\mathbb{N}}$ , we usually ignore the infinite blanks after the significant part of the text, thus  $\mathcal{L}(M) = \# \{0, 1\}^*$ .

Multiple definitions exist for Turing machines, we will prove the equivalence of the definitions.

1.  $\Gamma = \{0, 1, \square, \triangleright\}$ : consider a machine  $M$  with alphabet  $\Gamma'$ , we take the tape of the machine  $M$ , for each cell, we create  $\lceil \log_2(|\Gamma'|) \rceil$  cells, we can create special states for each of the symbols of  $\Gamma'$  to recognize it after a sequence of the new symbols.
2. Multiple tape turing machine: consider a machine  $M$  with  $k$ -tapes, we can make a new machine that uses just one tape, and that is by using the following mapping

$$(\mathbb{Z}^k)^{\mathbb{N}} \rightarrow \mathbb{Z}^{\mathbb{N}}$$

$$\left( (a_i^n)_{i=1}^k \right)_{n \in \mathbb{N}} \mapsto \left( a_{n \bmod k}^{\lfloor n/k \rfloor} \right)_{n \in \mathbb{N}}$$

3. “Stay” movement: the machine that has a Stay movement would work as follows, consider  $\delta(q_i, c) = (q_j, c', S)$ , we can set it as two steps  $\delta(q_i, c) = (q^*, c', R)$  then  $\delta(q^*, c) = (q_j, c, L)$ .

We talked earlier in the birth of Turing machines section about the reason for the existence of Turing machines, and that is, to find a solution for the decidability problem that states that is there a procedure to determine the truth of a statement. Now that we have the formal tool to define procedure, we can answer our question. We introduce the concept of decidability.

**Definition (Decidable Languages/Decider):**

Let  $\mathcal{L} \subseteq \Sigma^*$  be a language, we say that  $\mathcal{L}$  is decidable if there exists a Turing machine  $M$  such that

$$x \in L \Leftrightarrow M \text{ accepts } x$$

$$x \notin L \Leftrightarrow M \text{ rejects } x$$

We call  $M$  the decider of the language  $\mathcal{L}$ .

Here in the definition, we notice that  $M$  necessarily should halt for any input, since if it loops then  $M$  neither accepts nor rejects, so  $\neg(x \in L \vee x \notin L)$  which is contradictory by the law of excluded middle. We prove that there exists an undecidable language.

**Lemma (Typewriter Principle):** Let  $S$  be a set such that every element of  $S$  can be determined uniquely by a string in  $\Sigma^*$ , then  $S$  is countable.

*Proof.*  $\Sigma^* = \bigcup_{k \geq 0} \Sigma^k$  which is a countable union of countable sets thus it is countable,  $S$  can be determined uniquely by a string in  $\Sigma^*$  implies that there exists an injective map  $\varphi : S \rightarrow \Sigma^*$ , and since  $\Sigma^*$  is countable, then so is  $S$ . ■

**Theorem:** There exists an undecidable language.

*Proof.* Let  $D$  be the set of deciders and  $\mathcal{L}_D(\text{TM})$  be the set of languages decidable by a Turing machine. Notice that every Turing machine in  $D$  has a unique element in  $\mathcal{L}_D(\text{TM})$  which is its language, and thus  $\# \mathcal{L}_D(\text{TM}) \leq \# D$ , we can describe a Turing machine in a finite string given that it has finite states, symbols, and transitions in a unique way, thus by the Typewriter principle,  $D$  is countable, and so is  $\mathcal{L}_D(\text{TM})$ . We have that  $\Sigma^*$  is countable, by Cantor's theorem,  $\mathcal{P}(\Sigma^*)$  would be uncountable. Any map  $\varphi : \mathcal{L}_D(\text{TM}) \rightarrow \mathcal{P}(\Sigma^*)$  would never be surjective since  $\# \mathcal{L}_D(\text{TM}) < \mathcal{P}(\Sigma^*)$ , thus we conclude that there exists undecidable language. ■

Now we look for one of the most significant undecidable problems, the HALT language which is defined as follows

$$\text{HALT} = \{ \langle M, x \rangle \mid M \text{ halts on } x \}.$$

HALT being undecidable means that it is impossible to have a machine that predicts in prior for any Turing machine if it would halt on some given input.

**Theorem:** The language HALT is undecidable.

*Proof.* Suppose on the contrary that HALT is decidable. Then there exists a Turing machine  $H$  such that for any machine  $M$  and input  $x$ ,  $H$  accepts when  $M$  halts on  $x$  and rejects when  $M$  does not halt on  $x$ . Now we build another machine  $D$  which uses  $H$  as a subprogram.  $D$  would be defined as follows

```

1 - D(M):
2 - | if (H(M, M) halts)
3 - |   then loop
4 - |   else halt

```

Now we do  $D(\langle D \rangle)$ , if it halts, then  $H(M, M)$  did not halt, then  $D(\langle D \rangle)$  would not halt, else if it does not halt, then  $H(M, M)$  halts, then  $D(\langle D \rangle)$  would halt. Contradiction, thus  $H$  cannot exist. ■

A consequence of such a theorem is the following. Consider the set  $\mathcal{T}$  of all representations of Turing machines  $\mathcal{T} = \{ \langle M \rangle \mid M \text{ a Turing machine} \} \subseteq \Sigma^*$ .

**Theorem (Rice):** Let  $P \subseteq \mathcal{T}$  such that:

- $P \neq \mathcal{T}$  and  $P \neq \emptyset$ .
- $P$  is closed under the relation  $\forall \langle M_1 \rangle, \langle M_2 \rangle \in \mathcal{T}$ ,  
 $\langle M_1 \rangle \sim \langle M_2 \rangle \Rightarrow \mathcal{L}(M_1) = \mathcal{L}(M_2)$ .

Then the language  $P$  is undecidable, that is, there is no TM that can decide for any  $M$ ,  $\mathcal{L}(M) \in P$ .

Rice theorem states in essence that it is impossible to have a machine tells exactly that any machine would have a semantic property, that is, it is impossible to decide in prior if a property would be satisfied or not. The proof is somewhat really technical, so we give just the idea of the proof by reduction to the halting problem.

Suppose that we take some non-trivial property, like  $P$ : “The machine prints ‘Hello, World!’”. Suppose that there is a machine  $M_P$  such that it decides if a Turing machine satisfies  $P$ . We create a new machine  $D$  that runs

```
1 - D(M):
2 - | if (M halts)
3 - |   then print 'Hello, World!'
4 - |   else loop
```

thus using the decider  $M_P$ , one can deduce if the machine  $M$  halts, thus  $D$  is a machine that can decide if any machine halts or not, which is contradictory, given that the HALT language is undecidable. We can use this argument for any property. The proof uses a similar idea but in a more formal environment.

Even though the Turing machines seem limited by the existence of undecidable statements, but that is in nature what a procedure satisfies. We now express one of the most important statements about computation theory.

**Definition (Computable Function):** A function  $f : \mathbb{N}^k \rightarrow \mathbb{N}$ ,  $(n_1, \dots, n_k) \mapsto f(n_1, \dots, n_k)$ ,  $f$  is said to be computable if there exists a Turing machine  $M$  such that given  $\langle n_1, \dots, n_k \rangle$ , it computes the output  $\langle f(n_1, \dots, n_k) \rangle$  and halts.

Even though it seems like all functions are computable, it is indeed not the case. A known uncomputable function is the Busy Beaver function.

**Definition (Busy Beaver):** The Busy Beaver function  $BB(n)$  is defined as the maximum numbers of 1's a halting  $n$ -state Turing machine can write before halting, starting from a tape with all 0s.

This function is well defined since the number of machines is finite and thus the ones that halt at some point. We prove the following proposition.

**Proposition:** The function  $BB(n)$  is uncomputable.

*Proof.* Assume that  $BB(n)$  is computable, take  $m \in \mathbb{N}$  and define the following property  $P_m = \{\langle M \rangle \mid M \text{ halts on exactly } BB(m) \text{ ones}\}$ ,  $P_m$  satisfies the Rice theorem, thus  $P_m$  is undecidable, but if  $BB(n)$  is computable, then  $P_m$  is decidable, contradiction. Thus,  $BB(n)$  is uncomputable. ■

One of the most important statements in the theory of computation is the Church-Turing Thesis, which states in principle the following:

1. Any formalization of a computation machine is equivalent or less strong than a Turing machine.
2. The human mind is a Turing machine.

This statement is really strong, yet not provable, given its nature being mostly philosophical more than mathematical. This thesis is mostly accepted as true given that whenever we take any model for computation like  $\lambda$ -calculus, tag-systems and other computation machines, we can simulate them by a Turing machine, and we can simulate a Turing machine with them thus they are equivalent in power. Due to its meta-physical statement, it can give really weird statements such as the following

*“It is impossible to verify algorithmically that an AGI (Artificial General Intelligence) either understands language, is conscious, or any other property on it, except for specific cases.”*

This applies given that AGIs try to imitate the human mind which is by the Church-Turing thesis, a Turing machine equivalent in power.

For the proof of the existence of universal Turing machines, we would like an easier way to “program” Turing machines, so we start by giving a proof that if something can be written in a programming language, then it is equivalent to some Turing machine.

**Lemma:** Any Turing machine is equivalent to a code written in the following language

Consider the simple programming language, with the symbols  $\{0, 1, \square, \triangleright\}$  it has an infinite array  $T[i]$ ,  $i$  being a pointer at a location and it has the format:

*label: if (T[i] = c) then { cmds... }*

and we consider the following commands with it

| Command   | Description                              |
|-----------|------------------------------------------|
| set c     | sets the symbol at $T[i]$ to c.          |
| inc / dec | increment/decrement the pointer i.       |
| goto l    | jumps to label l.                        |
| hlt b     | halt the machine, accepting if $b = 1$ . |

*Proof.* We will now prove that we can create a machine for any code of this form. We make a naming convention, we take the labels to be  $l_k$  which is the  $k^{\text{th}}$  label and their subcommands to be  $c_k^j$  which is the  $j^{\text{th}}$  command in the  $k^{\text{th}}$  instruction. The set of states would be of the form  $Q = \{q_{k,j}\}_{k \in \mathbb{N}, j \in \mathbb{N}} \cup (q_a, q_r)$ , now we define how for each command how the transition function behaves. For simplification, we consider  $q_{\text{next\_ins}}$  to be  $q_{l_{k+1}}$ ,  $q_{\text{next}}$  to be either  $q_{c_k^{j+1}}$  if it is available, or  $q_{\text{next\_ins}}$  else. We define the transition function as follows

| Command       | Transitions                                                                                                                             |
|---------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| if (T[i] = c) | $\delta(q_l, x) = \begin{cases} (q_{\text{next\_ins}}, x, S) & \text{if } x \neq c \\ (q_{c_l^1}, x, S) & \text{if } x = c \end{cases}$ |
| set c         | $\delta(q_{c_l^k}, x) = (q_{\text{next}}, c, S)$                                                                                        |
| inc           | $\delta(q_{c_l^k}, x) = (q_{\text{next}}, x, R)$                                                                                        |
| dec           | $\delta(q_{c_l^k}, x) = (q_{\text{next}}, x, L)$                                                                                        |
| goto l        | $\delta(q_{c_l^k}, x) = (q_l, x, S)$                                                                                                    |

| Command | Transitions                                                                                                         |
|---------|---------------------------------------------------------------------------------------------------------------------|
| hlt b   | $\delta(q_{c_l^k}, x) = \begin{cases} (q_a, x, S) & \text{if } b = 1 \\ (q_r, x, S) & \text{if } b = 0 \end{cases}$ |

We proved that given a code, we can make a machine that simulates a code written in this simple language. Now we prove the converse, that given any machine, we can represent it by this language. Let  $M$  with states  $\{q_0, \dots, q_n, q_r, q_a\}$ , and suppose  $\delta(q_i, c) = (q_j, c', N)$ , we can translate this to a code as follows:

```

1 - state_qi: if (T[i] = c) {
2 -   set c'
3 -   <if N = R> inc
4 -   <if N = L> dec
5 -   goto state_qj
6 - }
```

and we add the two following instructions

```

1 - state_qr: hlt 0
2 - state_qa: hlt 1
```

We finished proving the equivalence of this simple language and the Turing machine. ■

Now we use this lemma and the Church-Turing thesis to deduce the following theorem

**Theorem:** Any modern programming language that can simulate the simple code we have written, is equivalent to a Turing machine.

*Proof.* Suppose that a language has the ability to define labels, have arrays, do conditions, read and write on the array, then it can write the simple language we defined in the previous lemma. So any Turing machine can be written inside the modern programming language, since the programming language is a computation formalism, then by the Church-Turing thesis, it will not be stronger than a

Turing machine, so it is exactly as powerful as a Turing machine can be since it can simulate it. ■

A small discussion on the Turing completeness may be brought up from the previous theorem, that some languages have the ability to simulate Turing machines and thus by the Church-Turing thesis they are Turing machines. This property of being able to simulate a Turing machine is called the Turing Completeness.

**Definition (Turing Complete):** A system  $S$  is Turing complete if for any given Turing machine  $M$ , there exists a program  $P$  (given as input), such that for every  $x$ , executing  $P$  on  $x$  yields the same result as running  $M$  on  $x$ .

Most modern languages are Turing complete, which allows the expressiveness of the model but also the halting problem. We show the existence of one of such Turing complete machines.

**Definition (Universal Turing Machine):** A universal Turing machine  $U$  is a machine that takes as input a description of a Turing machine  $M$ ,  $x$  as input of  $M$  and simulates  $M$  on  $x$ , that is:

$$\begin{aligned} U \text{ loops on } \langle M, x \rangle &\Leftrightarrow M \text{ loops on } x \\ U \text{ accepts } \langle M, x \rangle &\Leftrightarrow M \text{ accepts } x \\ U \text{ rejects } \langle M, x \rangle &\Leftrightarrow M \text{ rejects } x \end{aligned}$$

We can think of such a universal machine as a typical computer, and the description of a Turing machine  $M$  as a program that runs on the computer, and  $x$  are the inputs that go into the program. In such a way, the computer simulates a smaller machine that just runs the program. To prove such a statement, we need some theorems prior to simplify the writing of the code that encodes the universal Turing machine. To prove it, we will use what we proved in the previous section, that we can find a Turing machine for any program in a Turing complete language, and we just write a python code to create the universal Turing machine.

```
1 - def tm_sim(desc, inp):
2 -     Q = desc['states']
3 -     T = desc['tape']
```

```
4 -     d = desc['trans']
5 -
6 -     st = desc['q0']
7 -     acc = desc['qa']
8 -     rej = desc['qe']
9 -     tape = list(inp)
10 -
11 -     s = st
12 -     pos = 0
13 -     step = 0
14 -
15 -     while s not in (acc, rej):
16 -         step += 1
17 -         sym = tape[pos]
18 -         key = (s, sym)
19 -
20 -         if key not in tr:
21 -             s = rej
22 -             break
23 -
24 -         ns, ws, mv = tr[key]
25 -         tape[pos] = ws
26 -         if mv == 'L':
27 -             pos = pos - 1
28 -             if pos < 0:
29 -                 tape.insert(0, '_')
30 -                 pos = 0
31 -
32 -         elif mv == 'R':
33 -             pos = pos + 1
34 -             if pos >= len(tape):
35 -                 tape.append('_')
36 -
37 -         s = ns
38 -
39 -     return s == acc, tape
```

Which is indeed a simulator of a universal Turing machine, thus finishes our proof of its existence.

In the majority of this document, we give statements on languages but not on algorithms or problems. That is intentional, here we will discuss and justify why it is equivalent to talk about decision as languages, or computation done with a typical input/output.

Consider  $I$  be the set of all possible inputs of some problem and  $O$  be the set of outputs possible from the problem.

**Definition (Computational Problem):** A computation problem is a relation  $\Pi \subseteq I \times O$ , such that for each  $x \in I$ , zero or more  $y \in O$  satisfy  $(x, y) \in \Pi$ .  $\Pi$  is said to be computable if there exists a machine such that starting with the tape having  $\langle x \rangle$ , the computation stops at some point and writes  $\langle y \rangle$  on the tape.

**Definition (Decision Problem):** A decision problem is  $D \subseteq I$ , where for  $x \in I$ , checks for  $x \in D$ .  $D$  is said to be decidable, if there exists a machine such that given  $x$ , it accepts if  $x \in D$  and rejects if  $x \notin D$ .

**Theorem:** Let  $\Pi \subseteq I \times O$  be a computational problem, and define its decision version

$$D_\Pi = \{ \langle x, y \rangle \mid (x, y) \in \Pi \}.$$

then we have the following equivalence

$$\Pi \text{ is computable} \Leftrightarrow D_\Pi \text{ is decidable.}$$

We will do an informal proof-sketch for this theorem, given that it needs lots of construction that are technical.

*Proof.*

- $\Rightarrow$  Suppose that  $\Pi$  is computable, consider  $M_\Pi$  be the machine that computes  $\Pi$ , now we create a machine  $M_D$  such that for any input  $\langle x, y \rangle$ ,  $M_D$  runs  $M_\Pi$  as a subprogram and returns  $y'$ , then  $M_D$  checks if  $y = y'$ , thus  $M_D$  decides  $D_\Pi$  so  $D_\Pi$  is decidable.
- $\Leftarrow$  Suppose that  $D_\Pi$  is decidable, consider  $M_D$  be the machine that decides  $D_\Pi$ , create a new machine  $M_\Pi$  that takes some input  $\langle x \rangle$  and enumerate all  $y$ , and

runs  $M_D$  on  $\langle x, y \rangle$ , return on first  $y$  that satisfies the decider  $M_D$ , thus  $M_\Pi$  computes  $\Pi$ .  $\blacksquare$

Notice that it is significantly easier to turn a computation problem into a decision problem, while the inverse requires a lot more steps. This is insignificant in the study of computation, since we assume in theory however much time it needs as long as it is finite, and just about the possibility of computation, not its time.

To give an example of such a situation, consider the following problem, let  $a, b, c$  three variables, and we define the computational problem  $\Pi$  such that

$$(P) \quad \begin{cases} ax^2 + bx + c = 0 \\ x \geq 0 \end{cases}$$

We can directly reduce  $\Pi$  into a decision problem by considering the decision problem of the defined language  $\{ \langle a, b, c, x \rangle \mid x \text{ satisfies } (P) \text{ with parameters } a, b, c \}$ . Thus, anything proved on languages and deciders, works also on computations.

The Turing machine was initially created to discuss and prove the undecidability of some statements in mathematics. Yet it branched off to one of the most used tools to describe and model modern machines in computation power and it expanded to having multiple types. The one we described was the deterministic Turing machine.

This is the usual model that we use even in computers. The “determinism” comes from the fact that the transition function having only one next state to go to.

This is a different type of Turing machines, which can accept to move to multiple states at once, that can be described with the transition function being defined as

$$\delta : Q \times \Gamma \rightarrow \mathcal{P}(Q \times \Gamma \times \{L, R\})$$

We start by giving an example and then explaining how these types of machines can be interpreted, we consider the machine that given a word on the tape, it checks if it is in the language  $\mathcal{L} = \{ ww \mid w \in \Sigma^* \}$ . Here is the NDTM

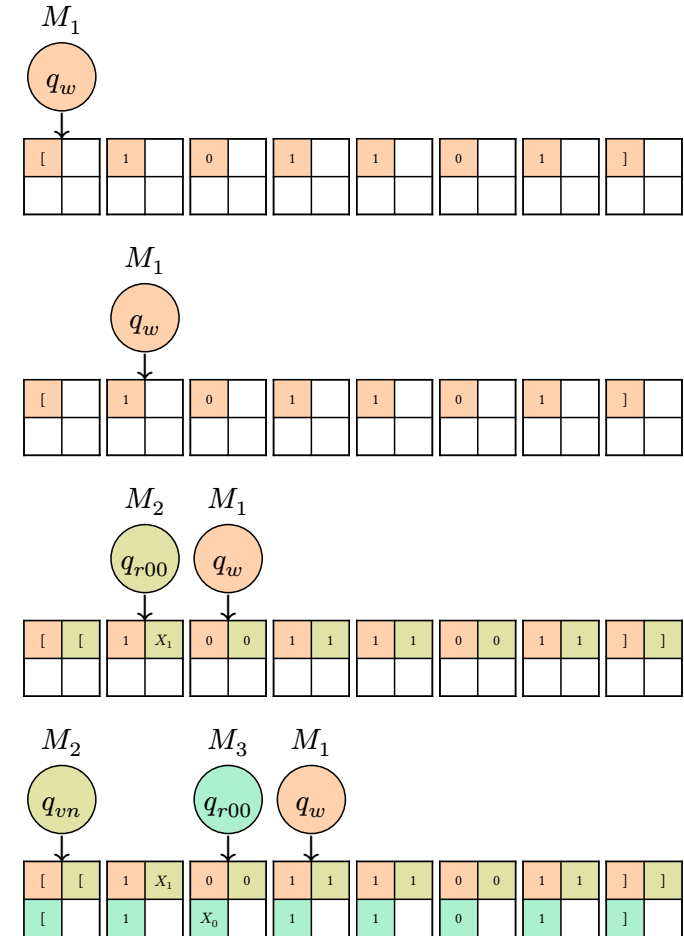
- The alphabet  $\Gamma = \{0, 1, X_0, X_1, [, ], \square\}$ .
- The states  $Q$ :
  - $q_w$ : walking state for start-points.
  - $q_{r00}, q_{r01}, q_{r10}, q_{r11}$ : remember one or zero state.

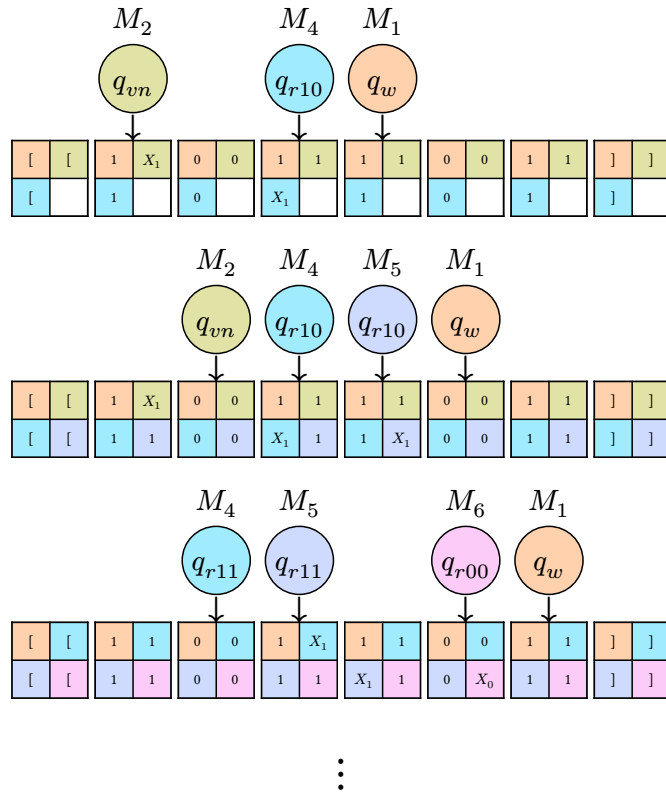
- $q_{mr0}, q_{mr1}$ : mark remembered state.
- $q_{nc0}, q_{nc1}$ : next character states.
- $q_{vs}, q_{vn}$ : verification states.
- $q_v, q_a, q_r$ : accept and reject states.
- The transition function  $\delta$ :

| $q \in Q$ | $c \in \Gamma$   | $\mathcal{P}(Q \times \Gamma \times \{L, R\})$                                                                            |
|-----------|------------------|---------------------------------------------------------------------------------------------------------------------------|
| $q_w$     | [                | • $(q_w, [, R)$                                                                                                           |
|           | ]                | • $(q_r, ], L)$                                                                                                           |
|           | 0, 1             | • $(q_w, c, R)$<br>• $\begin{cases} (q_{r10}, X_1, L) & \text{if } c=1 \\ (q_{r00}, X_0, L) & \text{if } c=0 \end{cases}$ |
| $q_{ri0}$ | $X_1, X_0$       | • $(q_{ri0}, c, L)$                                                                                                       |
|           | 1, 0             | • $(q_{ri1}, c, L)$                                                                                                       |
| $q_{ri1}$ | 1, 0             | • $(q_{ri1}, c, L)$                                                                                                       |
|           | $X_1, X_0, [$    | • $(q_{mri}, c, R)$                                                                                                       |
| $q_{mri}$ | $c = i$          | • $(q_{nc0}, X_i, R)$                                                                                                     |
| $q_{nc0}$ | 1, 0             | • $(q_{nc0}, c, L)$                                                                                                       |
|           | $X_1, X_0$       | • $(q_{nc1}, c, L)$                                                                                                       |
| $q_{nc1}$ | ]                | • $(q_{vs}, ], R)$                                                                                                        |
|           | $X_1, X_0$       | • $(q_{nc1}, c, L)$                                                                                                       |
|           | 1, 0             | • $\begin{cases} (q_{r10}, X_1, L) & \text{if } c=1 \\ (q_{r00}, X_0, L) & \text{if } c=0 \end{cases}$                    |
| $q_{vs}$  | 0, 1, $X_0, X_1$ | • $(q_{vs}, c, L)$                                                                                                        |
|           | [                | • $(q_{vn}, [, R)$                                                                                                        |

|          |            |                    |
|----------|------------|--------------------|
| $q_{vn}$ | $X_0, X_1$ | • $(q_{vn}, c, L)$ |
|          | ]          | • $(q_a, ], L)$    |

To make the table not too long, we used implicit rejections, that is, if there is a transition that is not available in the table, then it is a transition to the reject state. The workflow is quite easy, the machine starts with the tape of this form  $[w_1 \dots w_n]$  and accepts only if it can be written of the form  $ww$ . Here is a runtime of a non-deterministic Turing machine, notice that many machines were created in the process when running this non-deterministic machine.





As it can be seen here, it is quite difficult to simulate all of them, but the machine that will actually arrive to the accept state is  $M_5$  in this case, which can be easily verified by just moving through the states as the transitions become deterministic after we exit the walk state. Also notice that this language can be decided by the deterministic Turing machine too, as we can just iterate through each of the machines using one machine. An interesting theorem that comes from the Turing theory is that

**Theorem:** The set of languages of the deterministic Turing machines is the same as the one the language of non-deterministic Turing machines.

Even though both can recognize the language, the deterministic equivalent machine will add in the number of steps since they will not be done in “parallel”, we will talk more about this in a next section. Now that we defined a Turing machine, we can talk about complexity in a more formal way.

**Definition (Steps):** Let  $M$  be a (deterministic/non-deterministic) Turing machine, the number of steps is a function that takes  $n$  the number of non-empty symbols on the tape, and returns  $f(n)$  the number of usages of transitions to visit states until the machine halts.

We use the previously defined notations like  $O(\cdot)$ . It can clearly be seen for example in the inverter example that it takes  $O(n)$  steps, while the non-deterministic Turing machine for checking  $\mathcal{L} = \{ww \mid w \in \Sigma^*\}$  is  $\Theta(n^2)$  and its equivalent deterministic Turing machine is of order  $\Theta(n^3)$  since it has to go through the process  $2n$  times for each machine. Since we defined the measure of complexity, we can now define the classes of complexity. Usually, we talk about problems being in class **P**, **NP**, **NP-Complete**... etc. We consider a language to be DTM(resp. NDTM)-decidable in  $O(f)$  if there exists a deterministic(resp. non-deterministic) Turing machine that decides the language  $\mathcal{L}$  in  $O(f)$ .

**Definition (TIME/NTIME):**

$$\begin{aligned} \text{TIME}(f(n)) &= \{\mathcal{L} \text{ DTM-decidable in } O(f)\} \\ \text{NTIME}(f(n)) &= \{\mathcal{L} \text{ NDTM-decidable in } O(f)\} \end{aligned}$$

**Definition (P Class):**

$$\mathbf{P} = \bigcup_{k \geq 0} \text{TIME}(n^k)$$

which we usually take as the class of “efficient” algorithms, the philosophy for this choice comes from the fact that we usually accept that an algorithm of order  $O(n)$  or  $O(n^2)$  is quite efficient, also an algorithm calling another efficient algorithm

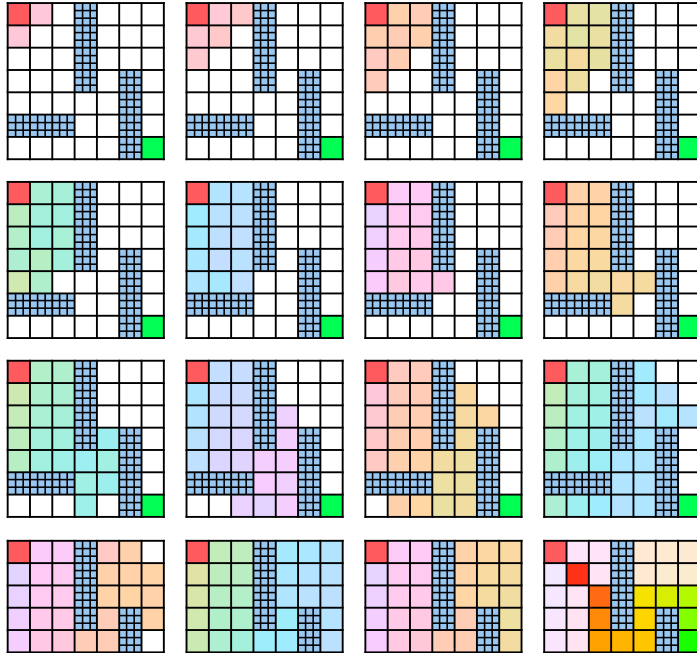
efficiently is also efficient, so for example  $O(n^3)$  calls an algorithm of  $O(n^2)$ ,  $O(n)$  times.

**Definition (NP Class):**

$$\mathbf{NP} = \bigcup_{k \geq 0} \mathbf{NTIME}(n^k)$$

which we usually take as the class of non-efficient algorithms, since it does not satisfy the criterion of efficiency per say of the class  $\mathbf{P}$ . A good property that comes from the class  $\mathbf{NP}$  is that any algorithm in  $\mathbf{NP}$  can be verified in polynomial time with a deterministic Turing machine, that is, there exists an algorithm  $\mathcal{A} \in \mathbf{P}$  to verify that the solution given by  $\mathbf{NP}$  is correct.

To explain the relation more, we will do an analogy, and then prove this statement. Consider a finite square grid, and a walker, that has to figure a path to arrive at a certain location in the grid. We simulate it below using a non-deterministic Turing machine, that at each point it takes all the possible paths, up, down, left, and right and we see the first one that solves the problem.



Every colored grid here represents a new machine that went in a different transition. Notice that in any setup that is like this, it will take at most  $n^2$  machines to find the path, and after finding it, we take the set of transitions of the machine that got to the accepted state first, and we make it into a deterministic machine. Thus, we can notice that a problem that is  $\mathbf{NP}$  can be verified in  $\mathbf{P}$  time.

**Definition (Certificate):** Let  $\mathcal{L} \in \mathbf{NP}$ , a certificate of an instance  $x$  is a string  $y$ , that given with  $x$  to a verifier, proves that  $x \in \mathcal{L}$ .

Consider the following SAT problem

$$\varphi = (x_1 \vee \overline{x_2}) \wedge (x_2 \vee x_3) \wedge (\overline{x_1} \vee \overline{x_3})$$

we take  $x = \langle \varphi \rangle$  its representation in the Turing machines, a certificate for this instance is an encoding of an assignment for  $\varphi$ , in this case  $y = \langle "x_1 = 1, x_2 = 0, x_3 = 1" \rangle$ . Or in the example above, given  $x$  an encoding of the obstacles, start and stop of the path finder algorithm,  $y$  would be the path that is drawn clearly in the last grid.

**Definition (Verifier):** A verifier for a language  $\mathcal{L}$  is a deterministic Turing machine  $V$  that takes inputs of the form  $\langle x, y \rangle$  and satisfies:

1. **Completeness:** if  $x \in L$  then there exists a certificate  $y$  with  $|y| \leq p(|x|)$  with  $p$  a polynomial such that  $V(x, y)$  accepts.
2. **Soundness:** if  $x \notin L$  then  $\forall y, V(x, y)$  rejects.
3.  $V \in \mathbf{P}$ .

**Theorem:**

$$\mathcal{L} \in \mathbf{NP} \Leftrightarrow \text{there exists a verifier } V \text{ for } \mathcal{L}.$$

*Proof.*

1.  $\Rightarrow$  Let  $\mathcal{L} \in \mathbf{NP}$ , let  $M_{\mathcal{L}}$  be a non-deterministic Turing machine in  $q(n)$  time, where  $q$  is a polynomial. Given that the states and symbols are finite, then at

each computation, there is at most a constant  $c$  of possible next moves. We can encode those choices of each possible path of computation in the non-deterministic paths into a string  $y$  that would have length  $O(q(n))$ .

Thus, we create the machine  $V$  such that for any input  $\langle x, y \rangle$ , it simulates  $M_{\mathcal{L}}$  on  $x$ , but instead of going through all the possible non-deterministic paths, it chooses the path given by  $y$ , which would run in a polynomial, the verifier accepts only if  $M_{\mathcal{L}}$  accepts  $x$  on the path given by  $y$ .

$V$  indeed verifies  $\mathcal{L}$  since if  $x \in \mathcal{L}$ , then there exists an accepting path  $M$  in  $x$  encoded by  $y$  of length less than  $\leq p(|x|)$  and  $V(x, y)$  accepts. And  $x \notin \mathcal{L}$  then there is no paths for acceptance so  $y$  would not exist.

2.  $\Leftarrow$  Let  $V$  be a verifier of  $\mathcal{L}$ . We construct the non-deterministic Turing machine  $M$  for  $\mathcal{L}$  as follows: for input  $x$ , it uses the non-deterministic behavior to write all possible strings  $y \in \Sigma^*$  such that  $|y| \leq p(|x|)$ , then in each of those run  $V(x, y)$ , accept if  $V$  accepts and reject otherwise.

$M$  runs in a polynomial time given that for each value of  $y$  it takes  $p(|x|)$  steps, and running  $V$  takes polynomial time in  $|x| + |y|$  which would be polynomial in  $|x|$ . If  $x \in \mathcal{L}$ , then there exists  $y \in \Sigma^*$  such that  $V(x, y)$  accepts so  $M$  accepts  $x$  and if  $x \notin \mathcal{L}$ , then  $y$  would not exist so all computation paths would reject thus  $M$  rejects  $x$ . Then  $M$  decides  $\mathcal{L}$  and since  $M$  is non-deterministic polynomial then  $\mathcal{L} \in \mathbf{NP}$ . ■

The central problem of complexity since its birth is the question of how different the  $\mathbf{P}$  and  $\mathbf{NP}$  classes are, that is

$$\mathbf{NP} \stackrel{?}{=} \mathbf{P}.$$

This question is among the seven Millennium Prize Problems, due to its sheer difficulty in proving or disproving it. Most theoretical computer scientist and mathematicians tend to believe that it is false, that is,  $\mathbf{P} \neq \mathbf{NP}$ . The absurdity from their equality is more clear when you take an example. A nice example is the path finding algorithm we developed earlier, by taking a big maze, knowing how many obstacles and where they are, also the start and end position, you can fairly quickly check if a given solution is correct, the absurdity when  $\mathbf{P} = \mathbf{NP}$  is the existence of an algorithm that just given those prior variables, can know what path to chose in the somewhat the same efficiency of checking the solution.

The problem I given before is known to be polynomial just by the breadth-first search algorithm, but the question still seems weird and tends to feel like it is an inequality, but for now it is not proved yet. Throughout the last century, an interesting equivalence has been set to simplify the search for this problem, which we will talk about in the next section.

**Definition (P-Computable Functions):** Let  $f : D \subseteq \Sigma^* \rightarrow \Sigma^*$ ,  $f$  is said to be computable in polynomial time, or **P-computable**, if there exists a Turing machine  $M$  such that, for any  $x \in D$ ,  $M$  halts with  $f(x)$  written on the tape, and runs in at most  $p(|x|)$  steps with  $p$  polynomial.

**Definition (Polynomial Reduction):** Let  $A, B \subseteq \Sigma^*$  two languages, we say that  $A$  reduces to  $B$  in polynomial time, denoted  $A \leq_p B$ , if there exists a polynomial-time computable function  $f : \Sigma^* \rightarrow \Sigma^*$  such that  $\forall x \in \Sigma^*, x \in A \Leftrightarrow f(x) \in B$ .

**Lemma:**

1. if  $B \in \mathbf{P}$  and  $A \leq_p B$  then  $A \in \mathbf{P}$ .
2. if  $B \in \mathbf{NP}$  and  $A \leq_p B$  then  $A \in \mathbf{NP}$ .

*Proof.*

1. Suppose that  $B \in \mathbf{P}$  and  $A \leq_p B$ , let  $f : \Sigma^* \rightarrow \Sigma^*$  the polynomial-time computable function that reduces  $B$  to  $A$ , denote the polynomial of the reduction  $f$ ,  $p(n)$ , and  $M_B$  the polynomial-time algorithm that decides  $B$ . We construct a machine  $M_A$  such as for any  $x \in \Sigma^*$ , compute  $f(x)$ , then run  $M_B$  on  $f(x)$  and accept or reject if  $M_B$  accepts or rejects and since  $x \in A \Leftrightarrow f(x) \in B$ , the number of steps that  $M_A$  takes is:  $p(|x|)$  for the calculation of  $f(x)$ , then running  $M_B$  on  $f(x)$  which takes  $q(|f(x)|)$  so the number of steps is  $p(|x|) + q(p(|x|)) = O(p(|x|))$  then  $M_A$  is a decider for  $A$  in polynomial time.
2. Suppose that  $B \in \mathbf{NP}$  and  $A \leq_p B$ , let  $f : \Sigma^* \rightarrow \Sigma^*$  the polynomial-time computable function that reduces  $B$  to  $A$ , since  $B \in \mathbf{NP}$  then there exists a polynomial-time verifier  $V_B$  for  $B$ , we construct  $V_A(x, y)$  as follows: compute  $f(x)$  and run  $V_B(f(x), y)$ , we get the result using the same reasoning as earlier. ■

**Definition (NP-Complete Class):** A language  $\mathcal{L}$  is said to be **NPC** (**NP-complete**) if  $\mathcal{L} \in \mathbf{NP}$  and  $\forall \mathcal{L}' \in \mathbf{NP}, \mathcal{L}' \leq_p \mathcal{L}$ .

The existence of **NP**-complete algorithms comes from a theorem proved by Cook-Levin Theorem.

**Theorem (Cook-Levin):** The language SAT

$$\text{SAT} = \{\langle \varphi \rangle \mid \varphi \text{ is a boolean formula in CNF} \\ \exists a \text{ assignment, } \varphi(a) = 1\}$$

*is an NP-Complete problem.*

A special case is the 3 – SAT, which given just 3 variables instead of arbitrary ones in the SAT language, is also an **NP**-complete problem. Multiple known **NP**-complete problems are known such as:

- Knapsack problem.
- Subset sum problem.
- Graph coloring problem.
- Hamiltonian path problem.
- Subgraph isomorphism problem.
- Boolean satisfiability problem (SAT).
- Travelling salesman problem.

Turing machines are a really useful formalism for computations which allowed us to see more complicated proofs on the how does algorithms work and practices.

## Chapter 4

# Complexity & Algorithms

Computational complexity theory, or complexity theory in short, is a theory made for classifying algorithms and computational problems by their time, memory, or other resources usage throughout their runtime which allows a clear objective comparison between algorithms performance in general.

It is easy to notice that for different kind of machines, the same algorithm will lead to different running times, and even sometimes in the same machine. To be objective, we will start by making a model for a universal machine, the Turing machine, that allows us to compare algorithms independent of the hardware they run on, thus giving us an objective measure for efficiency of algorithms.

# References

- [1] MICHEAL SIPSER, Introduction to the theory of COMPUTATION
- [2] STANFORD UNIVERSITY, Guide to the Myhill-Nerode Theorem : [https://web.stanford.edu/class/archive/cs/cs103/cs103.1252/guide\\_to\\_myhill\\_nerode](https://web.stanford.edu/class/archive/cs/cs103/cs103.1252/guide_to_myhill_nerode)
- [3] TAL MALKIN, Handout 3: The Myhill-Nerode Theorem and Implications : [https://www.cs.columbia.edu/~tal/3261/fall24/Handouts/3\\_Myhill\\_Nerode.pdf](https://www.cs.columbia.edu/~tal/3261/fall24/Handouts/3_Myhill_Nerode.pdf)